

Лабораторная работа - Линейное программирование в задачах оптимизации

Задача 1: Оптимизация производства электроники (Бизнес)

1. Математическая модель

а)

x_1 - количество смартфонов, выпускаемых за месяц (шт.)

x_2 - количество планшетов, выпускаемых за месяц (шт.)

б) Целевая функция

Максимизация общей прибыли:

$$P(x_1, x_2) = 8000x_1 + 12000x_2 \rightarrow \max$$

с) Ограничения

По ресурсам (неравенства - расход ресурса не должен превышать запаса):

- Процессорное время:

$$2x_1 + 3x_2 \leq 240$$

- Оперативная память:

$$4x_1 + 6x_2 \leq 480$$

- Аккумуляторы:

$$x_1 + 2x_2 \leq 150$$

Условия неотрицательности:

$$x_1 \geq 0, x_2 \geq 0$$

2. Построение функции Лагранжа

а) Приведение к канонической форме для минимизации

Функция linprog решает задачу минимизации, переводим максимизацию в минимизацию, меняя знак целевой функции:

Оригинал:

$$P(x) = 8000x_1 + 12000x_2 \rightarrow \max$$

Каноническая форма (для минимизации):

$$f(x) = -P(x) = -8000x_1 - 12000x_2 \rightarrow \min$$

Ограничения уже имеют вид $g_i(x) \leq 0$, где

$$g_1(x) = 2x_1 + 3x_2 - 240 \leq 0,$$

$$g_2(x) = 4x_1 + 6x_2 - 480 \leq 0,$$

$$g_3(x) = x_1 + 2x_2 - 150 \leq 0$$

б) Функция Лагранжа

Пусть u_1, u_2, u_3 - множители Лагранжа (для неравенств, $u_i \geq 0$). Тогда Лагранж для канонической задачи минимизации записывается как

$$L(x, u) = f(x) + u_1 g_1(x) + u_2 g_2(x) + u_3 g_3(x)$$

Подставляя f и g_i :

$$\begin{aligned} L(x_1, x_2, u) = & -8000x_1 - 12000x_2 \\ & + u_1(2x_1 + 3x_2 - 240) \\ & + u_2(4x_1 + 6x_2 - 480) \\ & + u_3(x_1 + 2x_2 - 150) \end{aligned}$$

Стационарные условия (необходимые условия ККТ) получаются дифференцирование по x_1, x_2 :

$$\begin{aligned} \frac{\delta L}{\delta x_1} = & -8000 + 2u_1 + 4u_2 + 1u_3 = 0, \\ \frac{\delta L}{\delta x_2} = & -12000 + 3u_1 + 6u_2 + 2u_3 = 0 \end{aligned}$$

Комплементарная нежесткость:

$$u_i \times g_i(x) = 0, \quad i = 1, 2, 3$$

Допуск (неотрицательность множителей и выполнение ограничений):

$$u_i \geq 0, g_i(x) \leq 0, x_1, x_2 \geq 0$$

с) Экономический смысл множителей Лагранжа u_i (Теневых цен)

Каждый множитель Лагранжа показывает маргинальную (теневую) цену ресурса: насколько изменится оптимальное значение целевой функции (максимальная прибыль) при небольшом увеличении запаса соответствующего ресурса на одну единицу.

- u_1 - теневая цена процессорного времени (часа)
Если увеличить запас процессорного времени на 1 час (RHS первого ограничения с 240 $\rightarrow$ 241), то при прочих равных оптимальных прибыль P^* увеличится примерно на u_1 рублей (в линейной аппроксимации)
- u_2 - теневая цена оперативной памяти (1 ГБ)
Увеличение запаса памяти на 1 ГБ даст прирост прибыли примерно u_2 рублей
- u_3 - теневая цена аккумулятора (1 шт.)
Увеличение числа доступных аккумуляторов на 1 шт. изменит прибыль примерно на u_3 рублей

3. Решение задачи на Python

```
el_prod_optimize.py X
el_prod_optimize.py > ...
1  import numpy as np
2  from scipy.optimize import linprog
3  import matplotlib.pyplot as plt
4
5  c = [-8000, -12000]
6
7  A_ub = [
8      [2, 3],
9      [4, 6],
10     [1, 2]
11 ]
12
13 b_ub = [240, 480, 150]
14
15 bounds = [(0, None), (0, None)]
16
17 result = linprog(c, A_ub, b_ub=b_ub, bounds=bounds, method='highs')
18
19 print("=== Electronics Production Optimization Task ===")
20 print(f"Status: {result.message}")
21 print(f"x1 (smartphones): {result.x[0]:.2f}")
22 print(f"x2 (tablets): {result.x[1]:.2f}")
23 print(f"Maximum profit: {-result.fun:.2f}")
```

Результат:

```
PROBLEMS  TERMINAL  OUTPUT  PORTS  DEBUG CONSOLE
Python + v [ ] [ ] ... [ ] [ ] X

PS C:\Users\Admin\Documents\Linear_Prog_Optimization> & "C:/Program Files/Python312/python.exe" c:/
• Users/Admin/Documents/Linear_Prog_Optimization/el_prod_optimize.py
=== Electronics Production Optimization Task ===
Status: Optimization terminated successfully. (HiGS Status 7: Optimal)
x1 (smartphones): 30.00
x2 (tablets): 60.00
Maximum profit: 960000.00
○ PS C:\Users\Admin\Documents\Linear_Prog_Optimization> [ ]
```

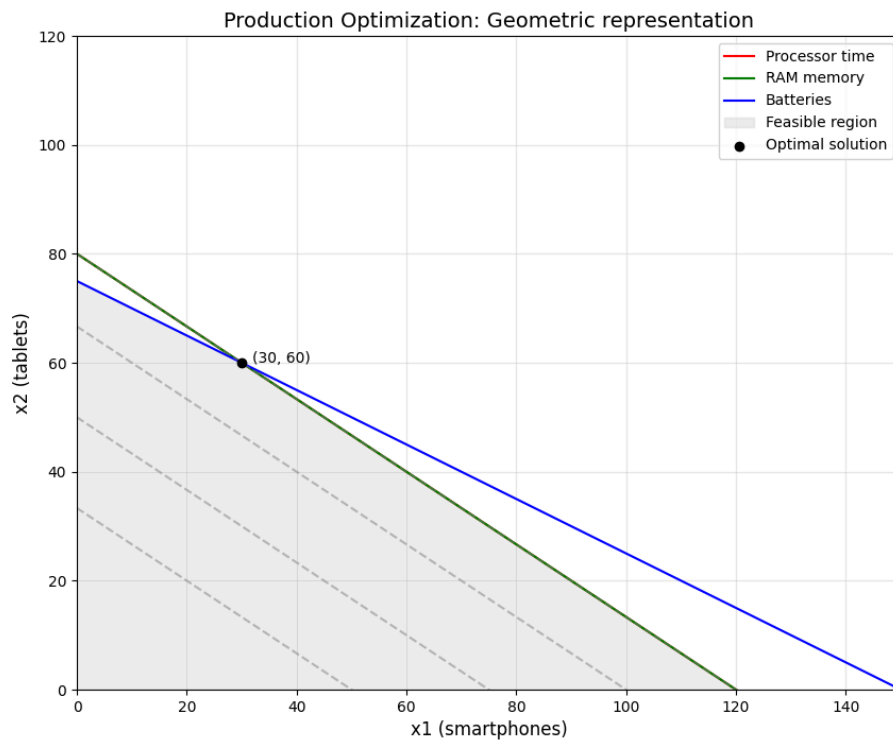
4. Геометрическая визуализация

```

el_prod_optimize.py X
el_prod_optimize.py > ...
1  import numpy as np
2  from scipy.optimize import linprog
3  import matplotlib.pyplot as plt
4  from matplotlib.patches import Polygon
5
6  c = [-8000, -12000]
7
8  A_ub = [
9      [2, 3],
10     [4, 6],
11     [1, 2]
12 ]
13
14 b_ub = [240, 480, 150]
15
16 bounds = [(0, None), (0, None)]
17
18 result = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method='highs')
19
20 print("=== Electronics Production Optimization Task ===")
21 print(f"Status: {result.message}")
22 print(f"x1 (smartphones): {result.x[0]:.2f}")
23 print(f"x2 (tablets): {result.x[1]:.2f}")
24 print(f"Maximum profit: {-result.fun:.2f}")
25
26 fig, ax = plt.subplots(figsize=(10, 8))
27
28 x1 = np.linspace(0, 150, 400)
29
30 x2_constraint1 = (240 - 2 * x1) / 3
31 x2_constraint2 = (480 - 4 * x1) / 6
32 x2_constraint3 = (150 - 1 * x1) / 2
33
34 ax.plot(x1, x2_constraint1, label='Processor time', color='r')
35 ax.plot(x1, x2_constraint2, label='RAM memory', color='g')
36 ax.plot(x1, x2_constraint3, label='Batteries', color='b')
37
38 ax.set_xlim(0, 150)
39 ax.set_ylim(0, 120)
40
41 vertices = np.array([
42     [0, 0],
43     [0, 75],
44     [30, 60],
45     [120, 0]
46 ])
47 polygon = Polygon(vertices, color='lightgray', alpha=0.4, label='Feasible region')
48 ax.add_patch(polygon)
49
50 x1_opt, x2_opt = result.x
51 ax.scatter(x1_opt, x2_opt, color='black', zorder=5, label='Optimal solution')
52 ax.text(x1_opt + 2, x2_opt, f"({x1_opt:.0f}, {x2_opt:.0f})", fontsize=10)
53
54 profit_levels = [400000, 600000, 800000, 960000]
55 for P in profit_levels:
56     x2_line = (P - 8000 * x1) / 12000
57     ax.plot(x1, x2_line, linestyle='--', color='gray', alpha=0.5)
58
59 ax.set_xlabel('x1 (smartphones)', fontsize=12)
60 ax.set_ylabel('x2 (tablets)', fontsize=12)
61 ax.set_title('Production Optimization: Geometric representation', fontsize=14)
62 ax.legend()
63 ax.grid(True, alpha=0.3)
64 plt.show()

```

Результат:



5. Анализ результатов

а) Интерпретация решения

- Оптимальный план:
Производить 30 смартфонов и 60 планшетов
- Максимальная прибыль: 960 000 руб. в месяц

б) Анализ использования ресурсов

Ресурс	Использовано	Доступно	Остаток	Статус
Процессорное время	$2 \times 30 + 3 \times 60 = 240$	240	0	Активное ограничение
Оперативная память	$4 \times 30 + 6 \times 60 = 480$	480	0	Активное ограничение
Аккумуляторы	$1 \times 30 + 2 \times 60 = 150$	150	0	Активное ограничение

Все ресурсы использованы полностью → все ограничения активны

Увеличение любого из ресурсов позволит увеличить прибыль

с) Чувствительность решения

- Если увеличить запас процессорного времени на 10 часов, то допустимая область расширится - можно будет произвести немного больше устройств, и прибыль увеличится
- Так как все ресурсы полностью использованы, каждый из них дефицитен
- Влияние каждого ресурса на прибыль можно количественно оценить по теневым ценам (множителям Лагранжа) - чем выше значение u , тем ценнее ресурс.

Задача 2: Оптимизация снабжения

2.1. Математическая модель

а) Переменные решения

- x_{11} - количество МТО (тонн), перевозимое со Склада 1 на Базу Альфа
- x_{12} - количество МТО (тонн), перевозимое со Склада 1 на Базу Бета
- x_{13} - количество МТО (тонн), перевозимое со Склада 1 на Базу Гамма
- x_{21} - количество МТО (тонн), перевозимое со Склада 2 на Базу Альфа
- x_{22} - количество МТО (тонн), перевозимое со Склада 2 на Базу Бета
- x_{23} - количество МТО (тонн), перевозимое со Склада 2 на Базу Гамма

б) Целевая функция

Минимизация общих транспортных расходов:

$$Z(x) = 8x_{11} + 6x_{12} + 10x_{13} + 9x_{21} + 7x_{22} + 5x_{23} \rightarrow \min$$

с) Ограничения

По запасам на складах (весь запас может быть отправлен):

$$\text{Склад 1: } x_{11} + x_{12} + x_{13} = 150$$

$$\text{Склад 2: } x_{21} + x_{22} + x_{23} = 250$$

По потребностям (каждая база должна получить требуемое количество):

$$\text{База Альфа: } x_{11} + x_{21} = 120$$

$$\text{База Бета: } x_{12} + x_{22} = 180$$

$$\text{База Гамма: } x_{13} + x_{23} = 100$$

Условия неотрицательности:

$$x_{ij} \geq 0 \text{ для всех } i = 1, 2, j = \text{Альфа, Бета, Гамма}$$

2.2. Проверка сбалансированности задачи

а) Вычисления

- Общий запас на складах: $150 + 250 = 400$ тонн
- Общая потребность баз: $120 + 180 + 100 = 400$ тонн

б) Вывод

Сумма запасов равна сумме потребностей ($400 = 400$) $\Rightarrow$ задача сбалансирована (закрыта). Это означает, что все поставки можно спланировать так, чтобы полностью израсходовать запасы и полностью удовлетворить потребности без ввода фиктивных поставщиков или потребителей.

2.3. С математическим анализом транспортной задачи ознакомился

2.4. Решение задачи на Python

```
supply_optimization.py X
supply_optimization.py > ...
1  import numpy as np
2  from scipy.optimize import linprog
3
4  c = [8, 6, 10, 9, 7, 5]
5
6  A_eq = [
7      [1, 1, 1, 0, 0, 0],
8      [0, 0, 0, 1, 1, 1],
9      [1, 0, 0, 1, 0, 0],
10     [0, 1, 0, 0, 1, 0],
11     [0, 0, 1, 0, 0, 1]
12 ]
13
14 b_eq = [150, 250, 120, 180, 100]
15
16 bounds = [(0, None)] * 6
17
18 result = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
19
20 print("=== Transportation Supply Problem ===")
21 print(f"Status: {result.message}")
22 x = result.x
23 print(f"x11 = {x[0]:.2f} tons")
24 print(f"x12 = {x[1]:.2f} tons")
25 print(f"x13 = {x[2]:.2f} tons")
26 print(f"x21 = {x[3]:.2f} tons")
27 print(f"x22 = {x[4]:.2f} tons")
28 print(f"x23 = {x[5]:.2f} tons")
29 print(f"Minimum total transportation cost: {result.fun:.2f} conv. units")
```



```

PROBLEMS    TERMINAL    OUTPUT    PORTS    DEBUG CONSOLE
● PS C:\Users\Admin\Documents\Linear_Prog_Optimization> & "C:/Program Files/Python312/python.exe" c:/Users/Admin/Documents/Linear_Prog_Optimization/supply_optimization.py
=== Transportation Supply Problem ===
Status: Optimization terminated successfully. (HiGHS Status 7: Optimal)
x11 = 0.00 tons
x12 = 150.00 tons
x13 = 0.00 tons
x21 = 120.00 tons
x22 = 30.00 tons
x23 = 100.00 tons
Minimum total transportation cost: 2690.00 conv. units
○ PS C:\Users\Admin\Documents\Linear_Prog_Optimization>

```

2.5. Визуализация сетевой диаграммы

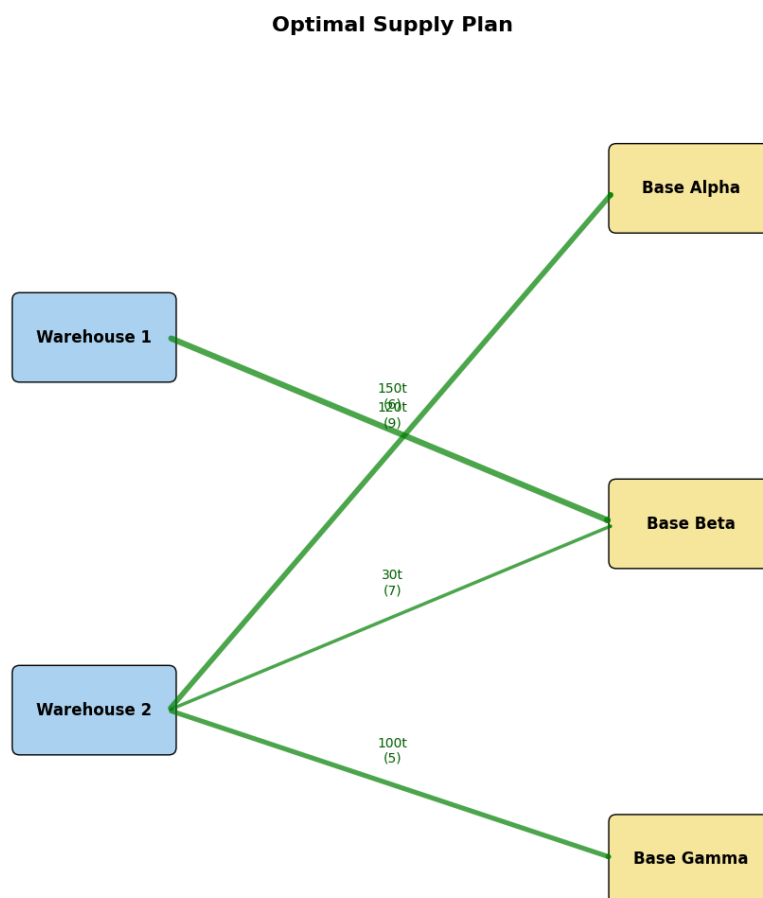
```
supply_optimization.py X
supply_optimization.py > ...
1 import numpy as np
2 from scipy.optimize import linprog
3 import matplotlib.pyplot as plt
4 from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
5
6 c = [8, 6, 10, 9, 7, 5]
7
8 A_eq = [
9     [1, 1, 1, 0, 0, 0],
10    [0, 0, 0, 1, 1, 1],
11    [1, 0, 0, 1, 0, 0],
12    [0, 1, 0, 0, 1, 0],
13    [0, 0, 1, 0, 0, 1]
14 ]
15
16 b_eq = [150, 250, 120, 180, 100]
17
18 bounds = [(0, None)] * 6
19
20 result = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='highs')
21
22 print("=== Transportation Supply Problem ===")
23 print(f"Status: {result.message}")
24 x = result.x
25 print(f"x11 = {x[0]:.2f} tons")
26 print(f"x12 = {x[1]:.2f} tons")
27 print(f"x13 = {x[2]:.2f} tons")
28 print(f"x21 = {x[3]:.2f} tons")
29 print(f"x22 = {x[4]:.2f} tons")
30 print(f"x23 = {x[5]:.2f} tons")
31 print(f"Minimum total transportation cost: {result.fun:.2f} conv. units")
32
33 fig, ax = plt.subplots(figsize=(14, 10))
34
35 warehouses = {'Warehouse 1': (2, 8), 'Warehouse 2': (2, 3)}
36 bases = {'Alpha': (10, 10), 'Beta': (10, 5.5), 'Gamma': (10, 1)}
37
38 for name, (x, y) in warehouses.items():
39     ax.add_patch(FancyBboxPatch((x-1, y-0.5), 2, 1, boxstyle="round,pad=0.1", fc="#AED6F1", ec="black"))
40     ax.text(x, y, name, ha="center", va="center", fontsize=12, weight='bold')
41
42 for name, (x, y) in bases.items():
43     ax.add_patch(FancyBboxPatch((x-1, y-0.5), 2, 1, boxstyle="round,pad=0.1", fc="#F9E79F", ec="black"))
44     ax.text(x, y, "Base " + name, ha="center", va="center", fontsize=12, weight='bold')
45
```

```

46 flows = [
47     ('Warehouse 1', 'Beta', 150, 6),
48     ('Warehouse 2', 'Alpha', 120, 9),
49     ('Warehouse 2', 'Beta', 30, 7),
50     ('Warehouse 2', 'Gamma', 100, 5)
51 ]
52
53 for wh, base, qty, cost in flows:
54     if qty > 0:
55         x1, y1 = warehouses[wh]
56         x2, y2 = bases[base]
57         arrow = FancyArrowPatch((x1 + 1, y1), (x2 - 1, y2),
58                                 arrowstyle='->', color='green',
59                                 linewidth=2 + qty / 60, alpha=0.7)
60         ax.add_patch(arrow)
61         ax.text((x1 + x2)/2, (y1 + y2)/2 + 0.3, f"{qty:.0f}t\n({cost})",
62                 ha="center", fontsize=10, color="darkgreen")
63
64 ax.set_xlim(0, 12)
65 ax.set_ylim(0, 12)
66 ax.set_aspect('equal')
67 ax.axis('off')
68 ax.set_title('Optimal Supply Plan', fontsize=16, fontweight='bold')
69 plt.tight_layout()
70 plt.show()

```

Результат:



2.6. Анализ результатов

а) Интерпретация решения:

- Используемые маршруты:
 - Склад 1 → Бета - 150 т
 - Склад 2 → Альфа - 120 т
 - Склад 2 → Бета - 30 т
 - Склад 2 → Гамма - 100 т
- Неиспользуемые маршруты: Склад 1 → Альфа, Склад 1 → Гамма
Они дороже, поэтому в оптимальном плане их нет
- Минимальная общая стоимость: 2690 у.е

б) Логистический анализ:

- Все базы получили требуемое количество МТО (Альфа 120, Бета 180, Гамма 100)
- Оба склада полностью разгружены (Склад 1 = 150, Склад 2 = 250)
- Основные поставщики:
 - База Альфа - Склад 2
 - База Бета - Склад 1 (основной)
 - База Гамма - Склад 2

в) Анализ устойчивости

- Если устойчивость маршрута Склад 2 → Гамма увеличится с 5 до 8 у.е, то поставки могут перераспределиться, и общие расходы вырастут.
- Если потребность Базы Альфа увеличить на 20 т, задача станет несбалансированной, потребуются либо увеличить запасы, либо доставить фиктивного поставщика.